

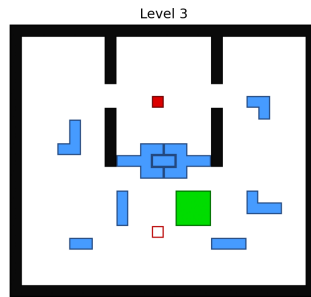
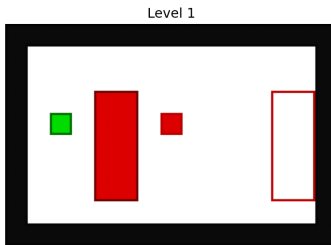
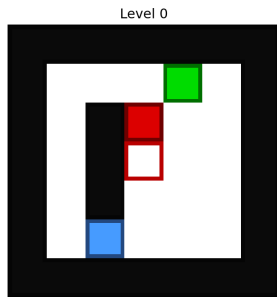
Промежуточный отчёт по PushWorld

Оптимизационный анализ пайплайна обучения

Иванкин Евгений, Сулиман Адам

Что такое PushWorld

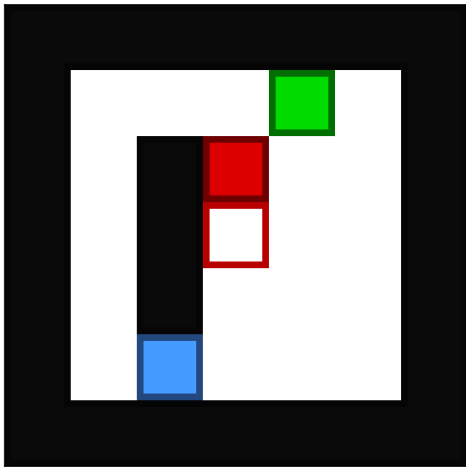
PushWorld benchmark examples



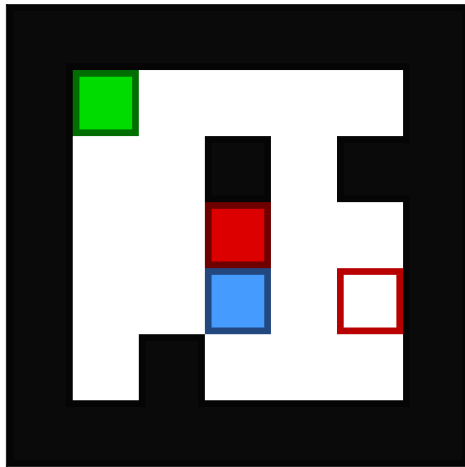
- PushWorld — planning benchmark: агент двигается по сетке и толкает объекты в целевые позиции.
- На сложных уровнях появляются инструменты, узкие проходы и длинные причинно-следственные цепочки.
- Для нас это удобный тест на то, где RL-пайплайн ломается по времени и по качеству.

Как выглядит решение в среде

DQN success
step=0 reward=0.00



PPO stochastic failure
step=0 reward=0.00



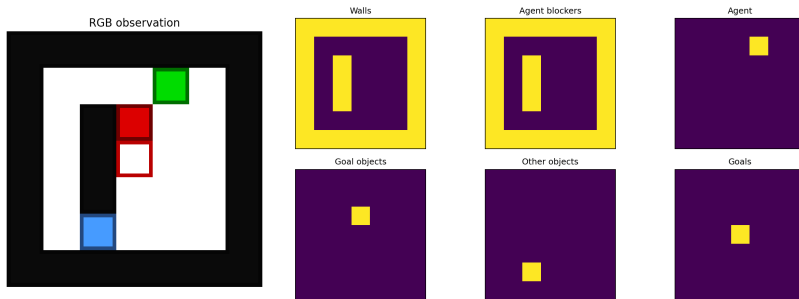
Бейзлайны слабые и медленные

- PPO на RGB оказался нестабильным даже на 5 простых головоломках.
- DQN на том же наборе вел себя устойчивее, но работал очень медленно.
- Это были короткие прогоны порядка 100–200k итераций: по качеству это ещё не финальный вердикт, но слабость бейзлайнов уже видна.
- Для этого checkpoint важнее именно системный сигнал: куда уходит время и какой FPS даёт каждая конфигурация.

Конфигурация	FPS	Успех
PPO, RGB, 5 головоломок	195	15%
DQN, RGB, 5 головоломок	8	40%

Что видит агент: RGB против planes

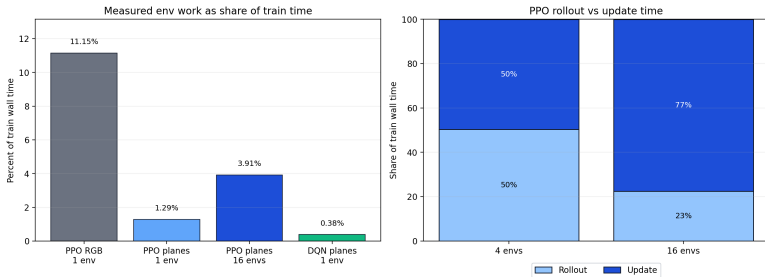
Structured plane observations



- RGB требует рендерить большую картинку и гонять большие тензоры через replay/CNN.
- Plane-наблюдения оставляют только полезную структуру состояния.
- Ключевые числа: размер состояния $58,800 \text{ B} \rightarrow 1,176 \text{ B}$; PPO $195 \rightarrow 586 \text{ FPS}$; DQN $8 \rightarrow 161.7 \text{ FPS}$.

После planes среда уже не главный bottleneck

PushWorld Bottleneck Summary



- PPO, RGB: среда занимает **11.15%** train time. PPO, planes: только **1.29%**.
- PPO, planes + 16 envs: **77.4%** времени уже уходит в update. DQN, planes: среда занимает всего **0.38%**, update — **76.0%**.
- Итог: сейчас главный bottleneck — не симуляция среды, а learner/update stack. Поэтому переписывать среду на GPU прямо сейчас преждевременно.

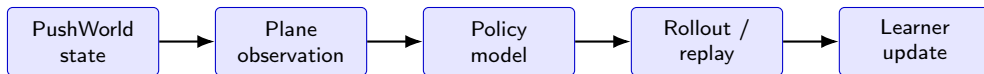
Ещё одна оптимизация: `torch.compile`

- Мы проверили уже не только representation-level оптимизации, но и model-side оптимизацию.
- `torch.compile` заметно ускоряет чистый forward pass.
- Но end-to-end training step ускоряется совсем слабо.
- Значит, оставшийся bottleneck шире, чем просто скорость инференса.

Batch	Метрика	Eager	Compile
256	Infer/s	1055.6	1643.6
256	Train/s	345.7	359.1
512	Infer/s	614.5	878.7
512	Train/s	201.6	208.3

Ускорение inference: **1.43x–1.56x**
Ускорение training: только **1.03x–1.04x**

Пайплайн, который мы ускоряем



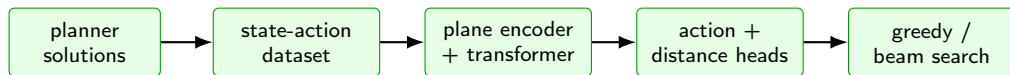
Уже ускорили

- RGB rendering → compact planes.
- Observation payload: 58,800 B → 1,176 B.
- PPO: 195 → 586 FPS; DQN: 8 → 161.7 FPS.

Следующие шаги для этого pipeline

- Goal-conditioned planes + relabeling [HER].
- Update profiling: dataloader, policy forward/backward, optimizer step.
- AMP, rollout length, minibatch size, update epochs, replay/buffer layout.
- Метрика: success/time и update seconds per 10^5 env steps.

Transformer policy pipeline

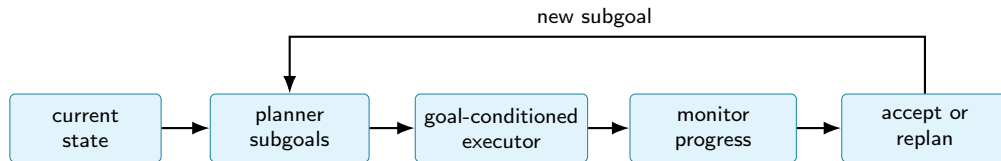


- Цель: ускорять не environment stepping, а supervised training и autoregressive search around policy.
- Метрики: action accuracy, greedy solve rate, beam solve rate, states/sec during search, GPU utilization.
- Основа: Sokoban transformer-policy + GPU rollout direction [Wheeler].

Transformer policy: конкретный план оптимизации

- 1 Dataset first: экспортировать planner traces и кэшировать plane tensors, чтобы training не ждал парсер/симулятор.
- 2 Model ablation: CNN-only no-history baseline $\rightarrow$ transformer over last k states $\rightarrow$ distance head.
- 3 Training optimization: фиксированный dataloader, AMP, `torch.compile`, larger batches; мерить samples/sec и validation.
- 4 Search optimization: сначала greedy, затем beam width 8/16/32; батчировать scoring всех beam candidates одним forward.
- 5 Cache policy inputs: не пересчитывать encoder для одинаковых states; хранить visited states и logits/distance.
- 6 Остановиться, если beam search time растёт быстрее solve rate; тогда оптимизировать ranking head, а не width.

Hybrid solver pipeline

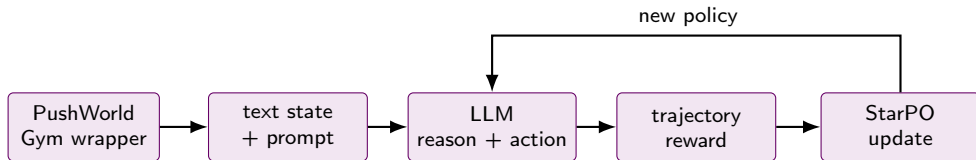


- Цель: заменить дорогой full search на planner skeleton + fast learned local execution.
- Метрики: subgoal success, replans per puzzle, planner time vs executor time, solved puzzles per minute.
- Близкая идея декомпозиции: planner задаёт структуру, learned follower исполняет локальные шаги [Learn to Follow].

Hybrid solver: конкретный план оптимизации

- 1 Offline subgoals: взять solution traces и нарезать на достижимые partial goals через каждые k действий.
- 2 Executor baseline: goal-conditioned CNN policy учится по imitation / relabeling на Level 0 subgoals.
- 3 Interface profiling: отдельно мерить planner call time, executor inference time, env stepping, replanning overhead.
- 4 Reduce planner calls: cache subgoal plans, replan only after timeout or distance-to-subgoal regression.
- 5 Learn ranker/value: если planner даёт несколько candidates, ранжировать их моделью вместо полного rollout search.
- 6 Ablation: planner-only, executor-only, hybrid without ranker, hybrid with ranker.

RAGEN pipeline: LLM-agent RL



- RAGEN trains LLM agents through multi-turn environment interaction: state text → reasoning/action → reward → policy update [RAGEN].
- PushWorld adaptation: Gym wrapper, deterministic text renderer, strict action parser, reward/success adapter.
- Baselines: fixed prompt without fine-tune on 256 episodes; then LoRA/StarPO on a small Level 0 split.

RAGEN pipeline: план оптимизации

- 1 Baseline profiling: wall time, tokens/sec, tokens/episode, env-step time, generation time, update time, GPU memory.
- 2 Prompt compression: verbose state vs compact ASCII planes; cap reasoning tokens; require one-token action answer.
- 3 Rollout batching: batch P initial states $\times N$ samples per state through vLLM; measure GPU utilization.
- 4 Runtime settings: reproduce paper's vLLM no-enforce-eager / graph-retention setup, then tune Ray workers and env placement.
- 5 Training cost: compare no-finetune, LoRA, full update by success/time and memory/time-to-first-improvement.
- 6 Rollout filtering: keep high-variance/high-signal prompt groups; measure time-to-target success, not just final reward.

RAGEN reports runtime/resource context, not a full throughput table: Figure 5 has Sokoban PPO total-time plots; Appendix C.2 gives H100/A100 + vLLM/Ray/FSDP settings; Appendix H compares LoRA resources [paper].

- [1] M. Andrychowicz et al. *Hindsight Experience Replay*. NeurIPS, 2017.
arxiv.org/abs/1707.01495
- [2] T. Wheeler. Серия материалов о Sokoban, sequence modeling и GPU rollout.
timallanwheeler.com/blog/category/sokoban/
- [3] *Learn to Follow: Decentralized Lifelong Multi-agent Pathfinding via Planning and Learning*, 2023.
arxiv.org/abs/2310.01207
- [4] Z. Wang et al. *RAGEN: Understanding Self-Evolution in LLM Agents via Multi-Turn Reinforcement Learning*, 2025.
arxiv.org/abs/2504.20073 github.com/ml-lab-nu/RAGEN



github.com/eivankin/pushworld